# 5300 PROJECT CONTENT

**Internet Secrets: Using Animations to Teach the Foundations of Network Security**

Developed by Captain Scott D. Russell

CYBSC 5300 Fall 2026

## PROJECT DESCRIPTION

My project fits into the visualization category: a website of short animated, interactive lessons (modules) that teach network security foundations to anyone! The modules build from basic understanding to more complex networking topics. Inspired by the endless Computer Based Training (CBT's) that I am mandated to complete every year as an active duty military member. I wanted to build my own CBT style training that, hopefully, is actually enjoyable and fun!

I developed the website in HTML, JavaScript and SVG and hosted it free on my GitHub Pages repository. I gave every module interactive buttons, so learners move through the animation at their own pace, and I added a short 5-question self-check quiz at the end of each module to test understanding. There is also a final quiz that covers all 4 lessons.

I re-purposed the animation code that I built for my Creative Component Presentation for CPRE 5990 as the template for this project. This drastically sped up the development of the animations themselves.

Below is a link to the current working version of the Internet Secrets project on my GitHub:

**https://captainrussell.github.io/Portfolio/projects/internet-secrets/**

## TARGET AUDIENCE

Anyone curious about how the internet protects their information: high school students, parents, and the general public. I wrote the modules at a high school level. I also have the additional duty of Cybersecurity Liaison for the Air Force Aerospace Studies Department at Iowa State, so it could be a fun lesson for my cadets as well.

## AUDIENCE SKILL LEVEL

No computer engineering, networking, or cyber background is required; learners only need a web browser and basic computer and arithmetic skills. Technical ideas like packets, encryption keys, hashing, and firewalls were taught in the module that needed them, using everyday comparisons (postcards, mixing buckets of paint, fingerprints, and a wall on fire) to avoid using any jargon or complex math/terminology. As an educator myself as an assistant professor, my goal when building lesson plans is to make it as digestible and understandable as possible for my students. I would much rather teach to a level where everyone can understand, than to teach at a much higher level where only a few of my students can grasp the concepts

effectively. Teaching isn't about presenting the most complicated topics, it's having the highest level of understanding across all students.

## LEARNING OBJECTIVES

After a learner views and interacts with the project, they should be able to do the following:

#1: Describe how a message is split into packets and passed from router to router.
#2: Explain how two people can safely agree on a shared secret while someone watches everything they send.
#3: Explain why websites store a password's fingerprint (hash) instead of the password itself, and why a long passphrase takes far longer to guess than a short password.
#4: Describe how a firewall decides which traffic to let in, and how an overly restrictive firewall can prevent legitimate traffic from passing through.

Each objective is met when a learner scores 80% (4 of 5) or higher on that module's quiz.

*Note: Objective 4 was narrowed during development from the initial proposal (the DDoS part was cut, as the proposal's fallback plan allowed). See Project Modifications. This was both an implementation challenge with so many visual elements flying around, and to avoid overloading and confusing the learner within one lesson.*

## RESEARCH METHODOLOGY

I started with the Wikipedia overview of network security [1] as a map of the field, then followed its topics to primary sources, so every fact a learner sees traces to a textbook, paper or standard. I picked four ideas a non-technical person meets every day (sending a message, the browser secure padlock, passwords, a network's front door) and placed each in the TCP/IP model [2], since the layer shows what an attacker can see and what protects it (Table 1). For each one I built the lesson and walked through it as a first-time learner, revising for clarity.

| Lesson | TCP/IP layer | Sources |
|---|---|---|
| 1. Postcards on the Internet | Network (IP routing) and transport (TCP numbering and resend) | [2], [3], [4] |
| 2. Mixing Secret Colors | TLS, between transport and application | [5], [6] |
| 3. Crack the Password | Application (login and password storage) | [7], [8], [9] |
| 4. The Fire Wall | Transport port numbers, checked at the network edge | [4], [10], [11], [12] |

TABLE 1. WHERE EACH LESSON SITS IN THE TCP/IP MODEL.

## RESEARCH FINDINGS

The following ideas and topics were validated from referenced sources to ensure accuracy and ease of understanding with clear metaphors and examples.

- **The network architecture:** IP promises no delivery or order [3]; TCP numbers the data so the receiver can spot a gap, ask again and reorder [4]. This became Lesson 1's numbered postcards and lost postcard animations (Figure 2).
- **A plain packet is a postcard:** Every router on the path, and anyone on the same Wi-Fi, can read it [2]. I had Eve read the whole message in Lesson 1 (Figure 3) to set up Lesson 2 to tie in ideas.
- **Strangers can agree on a secret in public.** Diffie-Hellman relies on a step that is easy to do and hard to undo [5], and TLS 1.3 uses a fresh exchange in every handshake to make the session keys [6]. I used paint mixing to show this (Figures 4 and 5) and added an optional math peek with real numbers (base 3, mod 17).
- **Hashes, salt, and length over complexity:** Sites should keep salted hashes, not passwords [7], [8]; one changed letter changes the whole SHA-256 output [9] (Figure 6). A thief can only guess and compare, so NIST favors long passphrases over forced symbols [8]. I built the crack-time bars to count (possible characters) to the power of (length) at an assumed 10 billion guesses a second, and counted the animal passphrase as lowercase letters only, 26 per letter with spaces not counted (Figure 7).
- *Password security soap box:* The only problem is that most websites use stupid password complexity requirements (uppercase, lowercase, special character, etc) instead of just a length requirement, leading learners to pick shorter, weaker passwords that are also harder to recall. Personally, I use KeePass, which is a local master password software to store strong and long complex passwords, but for a typical learner, a string of words is much more efficient as both security and memorization.
- **Deny by default, open only what is needed.** NIST recommends a default-deny firewall policy [10]. Ports are numbers in the transport header [4]: email between servers uses 25 [11] and SSH remote control uses 22 [12]. In Lesson 4 I used a burning wall that blocks everything until Bob opens each gate of the firewall.

## PROJECT CONTENT

I built a website of four short animated lessons and a final review, live at https://captainrussell.github.io/Portfolio/projects/internet-secrets/. I designed it so learners step through each animation at their own pace, tap the picture to act, and press Next when ready. I ended each lesson with a five-question check (4 of 5 meets the objective) and added a final review that asks 8 questions and shows an overall score. I used the same cast in every lesson: Alice (the sender), Bob (the receiver) and Eve (the eavesdropper). The use of Alice, Bob and Eve are classic in cryptography and were a nice touch and callback to traditional explanations of network security.

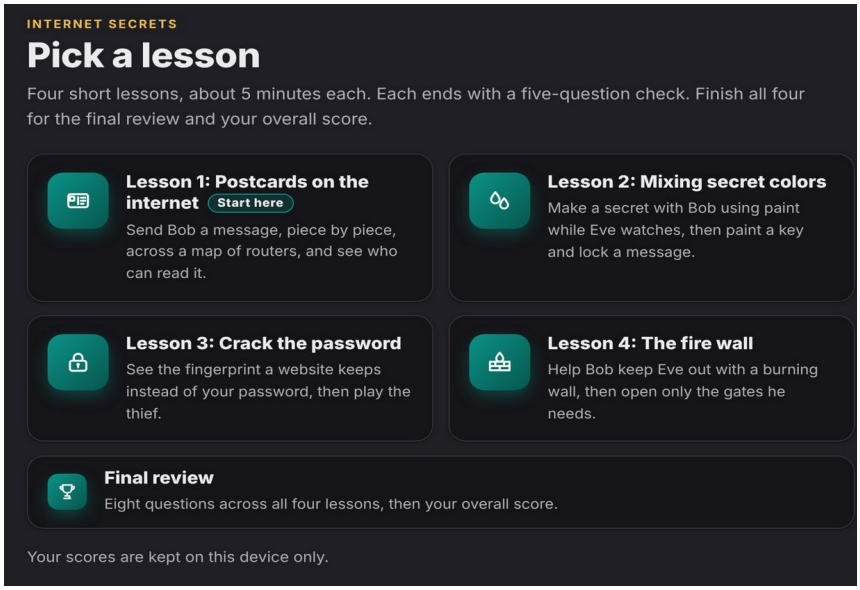



FIGURE 1. THE LESSON PICKER: FOUR LESSONS AND THE FINAL REVIEW.

## LESSON 1: POSTCARDS ON THE INTERNET (OBJECTIVE 1)

Alice picks a message, cuts it into numbered postcards and sends them. Routers pass them hop by hop, a jam destroys #1 while the rest reroute, Bob asks for #1 again, and the numbers put the message back in order. Finally Eve, on the café Wi-Fi, reads every postcard. I built it with 9 steps and 7 learner actions.




FIGURE 2. A JAM DESTROYS POSTCARD #1; BOB ASKS FOR IT AGAIN.

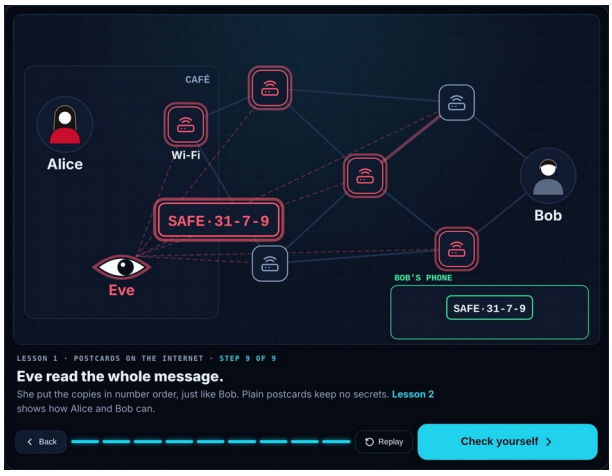



FIGURE 3. EVE READ EVERY POSTCARD: PLAIN PACKETS KEEP NO SECRETS.

## LESSON 2: MIXING SECRET COLORS (OBJECTIVE 2)

Alice and Bob each add a secret color to a public one, swap the mixes in the open, and add their own secret again, so both reach the same color that Eve cannot make. The shared color paints a key that locks a message, Eve's key does not fit, Bob's key unlocks it, and the https padlock appears. I built it with 17 steps and 15 learner actions, plus an optional math deep dive.




FIGURE 4. SAME COLOR ON BOTH SIDES; NO SECRET WAS EVER SENT.




FIGURE 5. EVE MIXES HER OWN KEY FROM WHAT SHE COPIED. IT DOES NOT FIT.

## LESSON 3: CRACK THE PASSWORD (OBJECTIVE 3)

Alice signs up, and Eve breaks into a site that saved plain passwords. A blender then turns each password into a fingerprint (a real SHA-256 hash), one changed letter gives a brand new fingerprint, salt makes the same password differ between sites, and the learner plays the thief against a stolen list. Crack-time bars end the lesson, and the learner builds a four-animal passphrase. I built it with 15 steps and 13 learner actions.

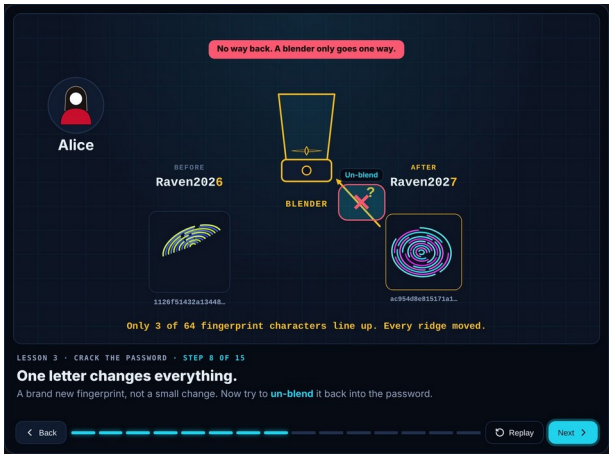



FIGURE 6. ONE LETTER CHANGED, A WHOLE NEW FINGERPRINT, AND NO WAY BACK.




FIGURE 7. CRACK TIME: A SHORT PASSWORD FALLS IN DAYS, THE PASSPHRASE DOES NOT.

## LESSON 4: THE FIRE WALL (OBJECTIVE 4)

Alice's email reaches Bob's shop, then Eve's remote-control packet takes it over. Bob builds a wall and sets it on fire, and now everything burns, including Alice's email. Bob opens gate 25 for email and keeps gate 22 for remote control shut, and then the learner sets the gates correctly. I built it with 15 steps and 7 learner actions.

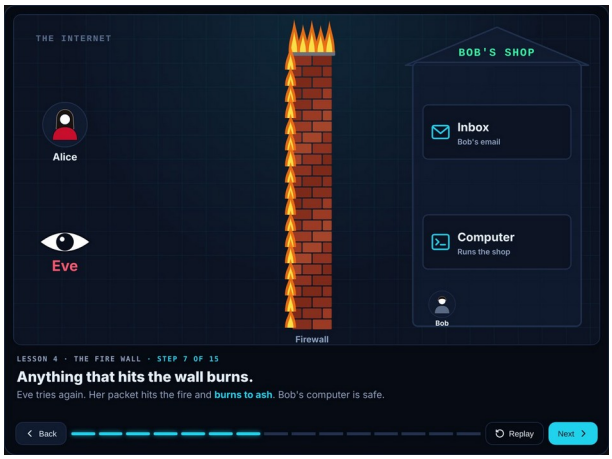



FIGURE 8. THE FIRE WALL BURNS EVERYTHING THAT HITS IT.

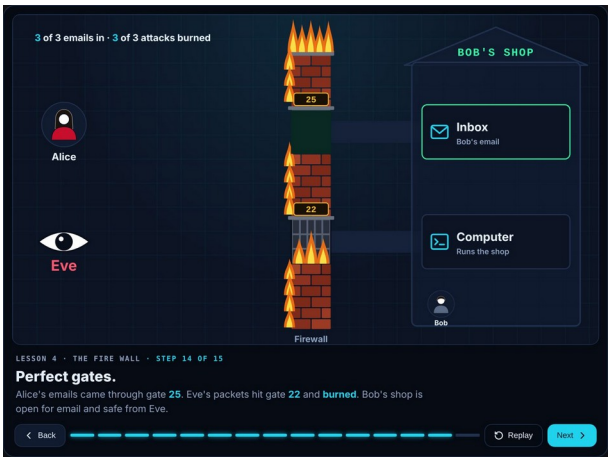



FIGURE 9. THE LEARNER'S GATES: 25 OPEN FOR EMAIL, 22 SHUT.

## PROJECT CONTENT DELIVERABLES AND SOURCE CODE

**Internet Secrets Project Page:**

- Item Description: Overview, learning objectives and the proposal PDF. The four lessons, their quizzes and the final review test.
- File name/path/link: https://captainrussell.github.io/Portfolio/projects/internet-secrets/
- Relevant information: Any modern browser, computer or phone. No login.

## DEPENDENCIES, RESOURCES & TOOLS

- A modern web browser. No install, framework, server or database; plain HTML, JavaScript and SVG. To run offline, any static web server will do (for example `python3 -m http.server`).

## REFLECTION

Reusing the animation engine from my CPRE 5990 slides gave me a working template early, and all four lessons were built by 7 November, ahead of schedule! The hardest part was keeping the lessons simple without making them wrong. Each comparison had to be checked against its source, for example the padlock meaning encrypted rather than safe. The other challenge was pacing. Cutting the DDoS section made the lesson shorter and more digestible. Next I will test with students who have no cyber background and use their quiz scores and

confusion points for any final changes to the lessons.
I plan to have my AFAS 3410 class of ~20 cadets/students as a captive audience to test the lessons!

## PROJECT MODIFICATIONS

- **Objective 4 narrowed:** I cut DDoS, as the proposal's fallback allowed, and limited the lesson to firewalls and ports 25 and 22 for simplicity. The cut of the DDoS section was due to a lack of clarity of animation and flow rather than a time limitation.
- **Ahead of schedule:** I have finished and published all four initial lessons and quizzes on my GitHub site.
- **More hands-on experience:** I gave nearly every step a learner action on the stage and replaced drag-and-drop with taps so it works on phones and with a keyboard without adding complex animations.
- **Name and home:** I renamed the project Internet Secrets and hosted it on my portfolio. I also moved the Creative Component slides linked in the proposal to my portfolio on GitHub, so everything is in one place: https://captainrussell.github.io/Portfolio/projects/daniels-cipher/slides/#6.

## REFERENCES


[1] Wikipedia contributors, "Network security," *Wikipedia, The Free Encyclopedia*. [Online]. Available: https://en.wikipedia.org/wiki/Network_security

[2] J. F. Kurose and K. W. Ross, *Computer Networking: A Top-Down Approach*, 8th ed. Hoboken, NJ, USA: Pearson, 2021.

[3] J. Postel, "Internet Protocol," IETF, RFC 791, Sep. 1981. [Online]. Available: https://www.rfc-editor.org/rfc/rfc791

[4] W. Eddy, Ed., "Transmission Control Protocol (TCP)," IETF, RFC 9293, Aug. 2022. [Online]. Available: https://www.rfc-editor.org/rfc/rfc9293

[5] W. Diffie and M. E. Hellman, "New directions in cryptography," *IEEE Trans. Inf. Theory*, vol. IT-22, no. 6, pp. 644–654, Nov. 1976.

[6] E. Rescorla, "The Transport Layer Security (TLS) Protocol Version 1.3," IETF, RFC 8446, Aug. 2018. [Online]. Available: https://www.rfc-editor.org/rfc/rfc8446

[7] R. Morris and K. Thompson, "Password security: A case history," *Commun. ACM*, vol. 22, no. 11, pp. 594–597, Nov. 1979.

[8] P. A. Grassi et al., "Digital identity guidelines: Authentication and lifecycle management," Natl. Inst. Stand. Technol., Gaithersburg, MD, USA, NIST SP 800-63B, Jun. 2017.

[9] National Institute of Standards and Technology, "Secure Hash Standard (SHS)," Gaithersburg, MD, USA, FIPS PUB 180-4, Aug. 2015.

[10] K. Scarfone and P. Hoffman, "Guidelines on firewalls and firewall policy," Natl. Inst. Stand. Technol., Gaithersburg, MD, USA, NIST SP 800-41 Rev. 1, Sep. 2009.

[11] J. Klensin, "Simple Mail Transfer Protocol," IETF, RFC 5321, Oct. 2008. [Online]. Available: https://www.rfc-editor.org/rfc/rfc5321

[12] T. Ylonen and C. Lonvick, Ed., "The Secure Shell (SSH) Transport Layer Protocol," IETF, RFC 4253, Jan. 2006. [Online]. Available: https://www.rfc-editor.org/rfc/rfc4253

[13] J. Mirkovic and P. Reiher, "A taxonomy of DDoS attack and DDoS defense mechanisms," *ACM SIGCOMM Comput. Commun. Rev.*, vol. 34, no. 2, pp. 39–53, Apr. 2004.


## ADDITIONAL INFORMATION

**Proposal feedback question about my slides:** they used no framework. I wrote a small keyframe animation engine in plain JavaScript and SVG with an AI assistant's help on the initial skeleton, and I reused the same engine for these lessons. The code is public in my GitHub repository.

**Use of AI tools:** as stated in my proposal, I used Claude (Anthropic, mostly the Opus 5.5 model) to help organize the code, generate the SVG drawings, and review this report for clarity from my research and direction. I chose the topics, references, sources and lesson design, and I take accountability that all content that has been created using an AI assistant is accurate and truthful.